LIC. EN CIENCIA DE DATOS

ALGORITMOS Y ESTRUCTURA DE DATOS I

Unidad 1-1: Repaso Programación Imperativa

Contenidista: Mg. Ing. Omar Gallo Haddad

ÍNDICE UNIDAD 1-1

Índice

1. Introducción 4
2. Tipos de Datos 5
3. Variables y Constantes 5
4. Operaciones Aritméticas 6
5. Operaciones de Relación 7
6. Operaciones Lógicas 7
7. Ingresar datos y mostrar por pantalla 7
8. Estructuras de Control 8
9. Registros 11
10. Arreglos 11
11. Matrices 12
12. Rutinas 13

MAPA DE LA UNIDAD

OBJETIVOS

- Repasar conceptos de tipos de datos básicos, estructuras de control, estructuras de datos fundamentales, arreglos, matrices, funciones, procedimientos, pasaje por valor y referencia y registros.

1. Introducción

Cuando programamos, describimos una secuencia de pasos que la máquina debe llevar a cabo a los fines de resolver un problema dado. Las formas y tipos de instrucciones que se encuentran a nuestra disposición varían según tipo de lenguaje y paradigmas de programación en cuestión a emplear.

En nuestro caso, haremos uso de un pseudocódigo con fines pedagógicos, que poseerá características propias de la llamada programación imperativa, de tal forma que nos permitirá trabajar con las características más comunes de este tipo de programación, independientemente de cuestiones específicas de lenguajes reales. Cuestiones tales como estructuras de control y de datos básicos son comunes en múltiples lenguajes reales, y los conceptos subyacentes que giran alrededor de los mismos son los mismos, pese a que cada lenguaje pueda tener características propias que otros no tengan o que sean diferentes. La idea de los conceptos aquí estudiados es que un alumno pueda transportar lo aquí aprendido de forma general a un lenguaje en particular llegado el caso.

Las instrucciones se escriben en un archivo y se ejecutan, una a una, de arriba hacia abajo. Seguirán ciertas reglas léxicas, sintácticas y semánticas que deberán ser respetadas a la hora de escribir las instrucciones que conforman el código en cuestión. En nuestro caso, el pseudocódigo que emplearemos, poseerá las propias.

A lo largo del código se podrán hacer comentarios, es decir agregar instrucciones que no sean ejecutadas por la máquina, y en donde el programador quiera dejar algún texto que no tenga que seguir las reglas léxicas, sintácticas o semánticas del código. Por ejemplo, esto puede usarse cuando se quiere dejar un comentario que aclare qué sucede en cierta parte del código, o algún comentario que el programador considere útil dejar allí. Para eso se hace uso de // o de /* */, en donde el primero sirve para hacer comentarios que abarquen una sola línea de código, y el segundo varias líneas. Por ejemplo:

```
…
// esto es un comentario de una sola línea y será omitido a la hora de ejecutarse el código
/* esto es un
comentario de múltiples
líneas y será omitido a la hora de ejecutarse
el código */
…
```

2. Tipos de Datos

Un tipo de dato es la característica que tiene un valor que determina las operaciones que se pueden realizar y cómo es almacenado en la memoria.

Los tipos que emplearemos son:

- entero: números enteros (ej.:_ 4, 123, -3, 0, etc.)
- real: números reales (ej.: 3.14, 5.1, 123.7)
- carácter: un solo símbolo (ej.: ´a´, ´&´, ´H´, ´3´, ´ ´, etc.)
- cadena: un conjunto de caracteres (“hola”, “123”, “hola mundo”)
- lógico/booleano: verdadero o Falso

Con esto, estableceremos los tipos de datos a los que responderán las distintas estructuras de datos con la que se trabajará en el código y la ejecución del programa.

3. Variables y Constantes

Una **variable** es un espacio en memoria, asociado a un nombre lógico y a un valor, con un tipo de dato particular. Su principal característica, es que su valor puede ir alterándose a lo largo de la ejecución del programa. Además, dependiendo del tipo de dato en cuestión, cambiará cómo se almacena el mismo en memoria.

El formato es:

NOMBRE_DE_VARIABLE: TIPO_DE_DATO

Por ejemplo:

```
…
sueldo: real
título: cadena [20] //el número entre paréntesis representa el tamaño máximo de la cadena
cantidad: entero
…
```

Una **constante**, a diferencia de una variable, poseerá el mismo valor durante toda la ejecución del programa.

El formato es:

NOMBRE_DE_LA_CONSTANTE = VALOR

Cuando trabajamos con constantes en nuestro pseudocódigo, el tipo de dato queda establecido en el momento de la asignación con el símbolo igual (=). Por ejemplo, al decir *A = 7.2*, automáticamente sabemos que *A* es real.

Es importante aclarar algunas cuestiones:

- El nombre de las variables tiene que ser un identificador compuesto por caracteres consecutivos sin espacios entre ellos, no pueden empezar con números ni pueden poseer caracteres extraños que no sean alfanuméricos como letras, números y guion bajo.
  Por ejemplo, los siguientes son **identificadores inválidos**: *nombre de la persona*, *123hola*, *%jaja%* y *código*. Ejemplos **válidos** son: *cantidad*, *cant*, *cantDeProductos*, *codigo*, *a*, *A*, *user1 y user_1*.
- Los caracteres en minúscula y mayúscula son distintos. Por ende los identificadores *a* y *A* son válidos y distintos entre sí.
- En caso de necesitar usar variables con un significado que no pueda resumirse a un identificador de una o pocas letras, es válido utilizar el **guión bajo (_)** para hacer las veces de espacio en el identificador. Por ejemplo, *cant_prod_limpieza* es un identificador válido. Otra forma es **camelizar** el nombre de la variable, es decir unir las distintas partes del identificador y que cada parte empiece con mayúscula para facilitar su lectura. Por ejemplo, *cantProdLimpieza*.
- Las constantes suelen escribirse completamente en mayúscula. El ejemplo más típico es el número pi, como *PI = 3.141516*

4. Operaciones Aritméticas

Disponemos de un conjunto de operaciones pare realizar las operaciones aritméticas básicas necesarias en la resolución de problemas:

- **+** (suma) -> afecta a los tipos **entero** y **real**
- **-** (resta) -> afecta los tipos **entero** y **real**
- ***** (multiplicación) -> afecta a los tipos **entero** y **real**
- **/** (división) -> afecta a los tipos **entero** y **real**
- **DIV** (división entera) -> afecta al tipo **entero**
- **RESTO** (resto de la división) -> afecta al tipo **entero**

Cabe destacar la diferencia entre /, DIV y RESTO.

Al realizar *7 / 5* el resultado es *1.4* (real) tomando a ambos o alguno de los dos como reales, mientras que *7 DIV 5* devuelve *1*, y *7 RESTO 5* devuelve *2*, si ambos números son enteros. En el caso del RESTO, considerando el algoritmo de la división tenemos que *7 = 5*1 + 2*, es decir que 2 es el resto de la división.

Esta aclaración es siempre importante tenerla presente, ya que en varios casos particulares será necesario usar con precisión alguno de ellos y no los otros.

5. Operaciones de Relación

En múltiples situaciones, como en el uso de las estructuras de control, será necesario crear expresiones en base a algún tipo de relación, cuyo resultado sea verdadero o falso.

La lista de operandos disponibles es:

- **<** (menor) -> afecta a los tipos **entero**, **real** y **carácter**
- **<=** (menor o igual) -> afecta a los tipos **entero**, **real** y **carácter**
- **>** (mayor) -> afecta a los tipos **entero**, **real** y **carácter**
- **>=** (mayor o igual) -> afecta a los tipos **entero**, **real** y **carácter**
- **=** (igual) -> afecta a los tipos **entero**, **real**, **carácter**, **cadena** y **lógico**
- **≠ o <>** (distinto) -> afecta a los tipos **entero**, **real**, **carácter**, **cadena** y **lógico**

Por ejemplo, la expresión *2 < 5* vale *verdadero* mientras que *2 > 5*, *falso*. Es importante tener en cuenta esto, ya que en base a los valores que se obtengan de estas expresiones, el código irá luego bifurcándose de acuerdo a lo establecido con las distintas estructuras de control empleadas.

6. Operaciones Lógicas

Son operaciones que se aplican a nivel tipo de dato lógico.

Los operandos con los que disponemos son:

- Y (conjunción) -> afecta al tipo de datos **lógico**
- O (disyunción) -> afecta al tipo de datos **lógico**
- NO (negación) -> afecta al tipo de datos **lógico**

Entonces, podemos construir expresiones que combinen varias relaciones entre sí. Por ejemplo, si necesitamos evaluar si una persona tiene una edad mayor a 18 y que su sexo sea masculino, podríamos tener *edad > 18 Y sexo = 'M'*. Entonces en caso de que ambos lados sean verdaderos, la conjunción será cierta. Si alguna de las dos fuera falsa, toda la conjunción daría *falso*. En el caso de la O, para que la disyunción sea falsa, es necesario que ambas sean falsas. En el caso del NO, la negación cambia el valor lógico dado. Es decir, si una variable o una relación vale *verdadero*, entonces la negación convierte a la misma automáticamente en *falso*.

7. Ingresar datos y mostrar por pantalla

- *Mostrar()* -> imprime por pantalla el mensaje que establezcamos entre paréntesis.

- *Ingresar()* -> guarda en la variable que se indique entre paréntesis el contenido que el usuario ingrese por teclado.

Ejemplo:

```
VARIABLES: // aquí declaramos las variables involucradas en el código
        cantidad: entero
        precio, total: real
        codigo: cadena[30]

INICIO //aquí inicia el programa

        Mostrar("Ingrese el código de producto")
        Ingresar(codigo) //ingresado por el usuario por teclado

        Mostrar("Ingrese cantidad de productos a comprar")
        Ingresar(cantidad)

        Mostrar("Ingrese el precio del producto")
        Ingresar(precio)

        total <- cantidad * precio

        Mostrar("Del producto ", codigo, " se compró un total de: ", total, " pesos")

FIN //aquí finaliza el programa
```

Algunos comentarios:

- Los espacios entre instrucción e instrucción no afectan la ejecución del programa. Podría haber muchas o ninguna línea entre cada instrucción y el código sería igualmente válido
- La **asignación (<-)** se interpreta de izquierda a derecha. El valor de lo que se evalúa a la derecha se asigna a la variable que está en la izquierda.
- En el *Mostrar()*, se interpreta que el valor de las variables código y total son automáticamente convertidos a su equivalente en cadena y concatenados a las cadenas que ya se encuentran explícitamente ubicadas dentro del *Mostrar()*. Es decir que una posible impresión por pantalla de la ejecución de este programa podría ser: "*Del producto AD45XF se compró un total de 14551.3 pesos*".

8. Estructuras de Control

Las estructuras de control permitirán alterar el flujo del programa generando bifurcación e iteración según distintas decisiones y repetir incluso una acción múltiples veces.

- Condicional Simple
  Si una condición es verdadera, entonces se ejecuta lo que se encuentra dentro del condicional, y luego se continúa donde este termina.

  Por ejemplo:

```
…
Si edad > 18 entonces
        Mostrar("Usted es mayor de edad")
fSi
… // y luego se continua desde aquí
```

- Condicional Alternativo

  En caso de que la condición sea verdadera, se ejecuta el código que se encuentra dentro del primer bloque de la estructura. De lo contrario (falso), se accede al bloque alternativo. Finalizada la ejecución de uno u otro bloque, el flujo continúa desde dónde termina todo el condicional (no se pasa de un bloque a otro).

  Por ejemplo:

```
Si estado = "activo"  entonces
        Mostrar("Usted puede acceder")
sino
        Mostrar("Usted no puede acceder")
fSi
```

- Condicional Anidados

  Describe a un condicional dentro de otro.

  Por ejemplo:

```
Si resultado = "victoria" entonces
        puntos <- 3
Sino
        Si resultado = "empate" entonces
                puntos <- 1
        Sino // por defecto, la única opción que queda es "derrota"
                puntos <- 0
        fSi
fSi
```

  Se podría haber explicitado *Si resultado = "derrota" entonces*, pero no era necesario puesto que solamente hay 3 opciones, y en caso de no cumplirse las primeras dos, entonces se terminará entrando en el último bloque del condicional.

- Mientras

  El código dentro del *Mientras* se repetirá hasta tanto y en cuanto la condición siga siendo verdadera. Es importante asegurarse de que dentro de la estructura de control algo siempre cambie de tal forma que en algún momento la condición se haga falsa, ya que de lo contrario el ciclo nunca terminará y nuestro programa quedará en un ciclo o loop infinito.

  Por ejemplo:

```
Ingresar(edad)
Mientras edad < 18 entonces
        Mostrar("Por favor, ingrese una edad mayor a 18")
        Ingresar(edad)
fMientras
```

Se podría considerar que este código funciona como una validación al ingreso de datos. Si el usuario hubiese ingresado una edad menor a 0, el Mientras forzaría al usuario a tener que ingresar una edad mayor o igual a 18. De lo contrario, el bucle nunca terminará.

- Repetir

  La diferencia con el *Mientras*, es que el *Repetir* se ejecuta siempre al menos una primera vez, y al final de este se evalúa la expresión dentro del *Hasta que*. Para que el bucle termine, es necesario que el valor de dicha expresión sea verdadera (al revés de lo que sucedía con el *Mientras*).

  Por ejemplo:

```
Repetir
        Mostrar("Por favor, ingrese una edad mayor a 18")
        Ingresar(edad)
Hasta que edad > 18
```

  Al igual que con el *Mientras*, es necesario asegurarse de que dentro del *Repetir* cambie algo que eventualmente haga que la expresión del *Hasta que* sea verdadero, para así evitar caer en un bucle infinito.

- Para

  Permite ejecutar código repetidamente como lo hacen el Mientras y el Repetir, pero permitiendo ir cambiando el valor de una variable desde un valor inicial hasta uno final concreto. Por defecto, estos incrementos se hacen de uno en uno y de forma ascendente.

  Ejemplo:

```
Para num <- 1 hasta 10 hacer
        Mostrar(num)
fPara
```

  En este caso, este código termina imprimiendo por pantalla todos los valores desde el 1 hasta el 10 incluido.

9. Registros

Nos permite tener una misma variable que contenga múltiples campos de distintos tipos de datos cada uno. Por una cuestión de simpleza y facilidad a la hora de trabajar, si manejamos el concepto de Empleado, sería más fácil definir un tipo de dato como el siguiente:

```
T_REG_EMPLEADO = Registro
        Nombre: cadena
        Apellido: cadena
        Legajo: cadena
        Sueldo: real
        …
fReg

…

empleado: T_REG_EMPLEADO
```

Entonces, para acceder a cada campo del registro, usamos el **operador punto (.)**

Si queremos ver el nombre del empleado, podemos hacer *Mostrar(empleado.nombre)*

10. Arreglos

En un arreglo, disponemos de la posibilidad de poder trabajar con un conjunto finito de elementos del mismo tipo de dato, los cuales se encuentran dispuestos de forma consecutiva uno a continuación del otro, y que poseen un mismo nombre en común.

La posición de estos elementos está numerada empezando por el número 1, y es posible acceder a cada elemento en forma directa usando los **corchetes ([ ])** e indicando la posición del arreglo.

El arreglo tiene un tamaño máximo definido a la hora de la declaración de las variables, y es importante tener siempre en cuenta que es necesario evaluar al arreglo entre el valor **1** y el **máximo** del arreglo. De lo contrario se estaría accediendo a posiciones que exceden estos rangos, provocando distintos tipos de error de desbordamiento de memoria.

Empleado, sería más fácil definir un tipo de dato como el siguiente:

```
MAX = 5
arr: arreglo de MAX enteros
…
Arr[1] = 64
Arr[2] = 32
Arr[3] = 11
```

```
Arr[4] = 5
Arr[5] = 97
…
```

El resultado de esta operatoria sería un arreglo como el siguiente:

64	32	11	5	97
1	2	3	4	5

Cabe destacar que las 5 posiciones de este arreglo están reservadas en memoria independientemente de si luego se hará uso de ellas en la ejecución del programa o no.

Dado este ejemplo, evaluar el arreglo en la posición 6 en adelante, sería erróneo, puesto que este está definido entre las posiciones 1 y 5.

11. Matrices

Implica llevar la noción de arreglos, pero ahora a dos dimensiones, es decir, teniendo la noción de filas y columnas. El primer valor dentro de los corchetes representa a la fila, y el segundo a la columna.

```
MAX = 3
mat: matriz de [MAX, MAX] enteros
…
mat[1, 1] = 23
mat[1, 2] = 88
mat[1, 3] = 17
mat[2, 1] = 4
mat[2, 2] = 6
mat[2, 3] = 11
mat[3, 1] = 41
mat[3, 2] = 8
mat[3, 3] = 5
…
```

El resultado de este código sería una matriz como la siguiente:

23	88	17
4	6	11
41	8	5

12. Rutinas

- Pasaje de parámetros
  - Por **valor**: se genera una copia del parámetro que se utiliza dentro de la rutina, por lo todo cambio que sufra esta copia no se verá reflejada en el parámetro original una vez terminada la ejecución de la rutina
  - Por **referencia**: en vez de generarse una copia del parámetro, se trabaja con una referencia a la dirección de memoria del parámetro original, por lo que todo cambio que se realice dentro de la rutina, terminará afectando al parámetro original. Es decir, las modificaciones del parámetro persistirán luego de finalizada la rutina.
- Función

  Código que puede ser invocado reiteradamente en múltiples lugares, que posee un nombre identificatorio junto a un número de parámetros, y que devuelve un valor único.

```
Función esPar (val num: entero): lógico
        Devolver (num RESTO 2 = 0)
fFunc
…
resp1, resp2: lógico
…
resp1 <- esPar(4) // verdadero
resp2 <- esPar(3) // falso
…
```

Por lo general, se opera de tal forma que las funciones manejen siempre parámetros por valor, y que devuelvan exactamente un valor, como en las funciones matemáticas. No se busca hacer uso de ingreso de dato por teclado o impresión de mensajes por pantalla

- Procedimiento

  Acepta tanto parámetros por valor y referencia, e incluso no necesita tener parámetro alguno. No devuelve un valor como hacen las funciones, sino que en todo caso puede disponer de alguno o varios parámetros por referencia.

```
Proced ingresarNum (ref num: entero)
        Repetir
                Mostrar("Ingrese un entero positivo")
                Ingresar(num)
        Hasta que num > 0
fFunc
…
a: entero
…
ingresarNum(a)
Mostrar(a) // se imprimirá por pantalla un número entero positivo ingresado por teclado
…
```